Unidad 2

Calculo lambda (Alonzo Church)

El cálculo lambda es un sistema inventado por Alonzo Church en los años 1930. Sirve para explicar qué significa calcular usando solo funciones. En este sistema todo es una función. Una función se escribe así: $\lambda x. x+1$, significa "toma x y devuelve x+1". Para calcular se cambia la x por un valor: si aplicas esa función a 5, queda 5+1. Ese cambio se llama reducción beta. Repitiendo cambios se llega al resultado. Church usó este sistema para mostrar que hay problemas que no se pueden resolver con ningún algoritmo. Además, se vio que el cálculo lambda puede calcular lo mismo que una máquina de turing. Esa idea se llama tesis de Church-Turing y marca los límites de lo que una computadora puede hacer. por eso es la base de lenguajes funcionales como Haskell y de las funciones lambda en Javascript o Python.

El cálculo lambda es un sistema muy simple donde todo es función y calcular es sustituir; con esto, explica qué puede hacer una computadora y está en el origen de mucha programación actual.

Escaneado con CamScanner

Elixir y Erlang

Elixir: Es un lenguaje de programación funcional, concurrente y de alto nivel de propósito general que se ejecuta sobre la máquina virtual, la misma que utiliza Erlang.

Se construye sobre la plataforma de Erlang para heredar sus capacidades de concurrencia y robustez sobre la maquina virtual. Elixir es un lenguaje moderno que utiliza esa misma base técnica, pero con un enfoque en la ergonomia y la productividad del desarrollador.

Erlang

lenguaje de programación concurrente y funcional de propósito general, diseñado específicamente para construir sistemas distribuidos, de tiempo real y tolerancia a fallos

Su característica más distintiva es que la concurrencia es parte del propio lenguaje, no del sistema operativo.

Escaneado con CamScanner

Programación imperativa lenguaje que usa whatsapp

19 10 2025

Unidad: 2

Funciones de orden Superior (map, filter, fold / reduce). Sobre listas

Una función de orden superior es una función que recibe otra función como argumento o devuelve una función como resultado. Las funciones de orden superior permiten abstraer patrones de recorrido sobre listas, evitando escribir recursiones o bucles repetitivos.

map: transforma cada elemento de una lista.

map :: (a -> b) -> [a] -> [b]

filter: Selecciona los elementos que cumplen una condición.

filter :: (a -> Bool) -> [a] -> [a]

Fold / reduce: combina todos los elementos de una lista en un único valor.

foldl :: (b -> a -> b) -> b -> [a] -> b

Estas funciones aparecen en lenguajes funcionales como Haskell, en lenguajes multiparadigma como Python o JavaScript, y tienen equivalentes en programación lógica mediante predicados como maplist, include y foldl.

Las funciones de orden superior map, filter y fold / reduce constituyen la base del procesamiento de listas en programación funcional.

- Tienen equivalentes claros en programación lógi
- Permiten escribir código más declarativo, conciso y reutilizable.

Escaneado con CamScanner

Cuadro comparativo

	Prefija	Infija	Posfija
Definición	El operador va antes de los operandos. También se llama notación polaca.	El operador va entre los operandos. Es la notación algebraica habitual.	El operador va después de los operandos, También se llama notación polaca inversa (RPN)
Forma General	Op a b	a op b	a b op
Ejemplo (3+4)*2	*+342	(3+4)*2	34+2*
Paréntesis y Precedencia	No se necesita el orden lo da la posición	Necesarias, junto con reglas de precedencia y asociatividad.	No se necesitan
En prolog	Es la forma canónica de todo término. También hay operadores prefijos declarativos	Operadores XFX, YFX, como is, :- Y, .	Solo con operadores declarativos en programado (xf, yf)

Escaneado con CamScanner

4/11/2026

Conceptos Intervalo, operadores y Evaluación perezosa

Intervalo: Un intervalo es un rango de valores entre dos límites. En Haskell no existe un tipo llamado "intervalo": se representa con listas generadas por rangos

Sintaxis

[inicio .. fin]

Ejemplo:

[1..5] --[1,2,3,4,5]

Haskell llama a una función de la clase Enum llamada enumFromTo:

[1..5] == enumFromTo 1 5

Formas de escribir intervalos

[a..b]	Ejemplo: [1..5]
[a,b..c]	Ejemplo: [1,3..9]
[a..]	Ejemplo: [1..]
[a,b..]	Ejemplo [2,4..]

Un intervalo en Haskell es una lista generada por la clase Enum mediante la sintaxis [a..b]. No es un tipo especial: es una lista normal que puede ser infinita porque la evaluación perezosa solo produce los elementos que el consumidor demanda. El salto lo determina el segundo elemento en la forma [a,b..c] y para descender es obligatorio indicarlo.

Escaneado con CamScanner

Operador: Identificador formado exclusivamente por símbolos que representa una función y que puede usarse en notación infija.

Un operador es una función que se escribe entre sus dos argumentos.

1 + 2

Símbolos permitidos
! # $ % & * + . / < = > ? @ \ ^ | - ~ :

No debe contener: letras, números, paréntesis, corchetes ni llaves.

Precedencia: cada operador tiene un número del 0 al 9 más alto = se evalúa primero.

2 + 3 * 4 — 14 porque * (7) va antes que + (6)

Número	Operadores	Se agrupan
9	.	Derecha
8	^, **	Derecha
7	*, /, div, mod	Izquierda
6	+, -	Izquierda
5	:, ++	Derecha
4	==, <, >,	No se pueden agrupar
3	&&	Derecha
2	||	Derecha
0	$, $!	Derecha

Si no se agrupa nada: Precedencia y se agrupa por la

Escaneado con CamScanner

Asociatividad: cómo se agrupan los iguales. Solo importa cuando hay varios operadores del mismo número.

```
10 - 5 - 2      -- (10-5) - 2 = 3      (izquierda)
1 : 2 : [3]     -- 1 : (2 : [3]) = [1,2,3]  (derecha)
1 == 2 == 3     -- ERROR (No se puede)
```

¿Por qué `1 == 2 == 3` da error? Porque `1 == 2` da True y `True == 3` no tiene sentido.

Forma correcta

```
1 == 2 && 2 == 3    -- False
```

Un operador en Haskell es una función de dos argumentos escrita en medio. Su comportamiento depende de dos cosas: precedencia (número del 0 al 9, desde qué se evalúa primero) y asociatividad (desde cómo se agrupan los del mismo número). Las más importantes son `$`, `.`, `:`, `++`, `&&`, `||` y los de comparación. Las secciones como `(*2)` fijan un argumento y dan una función de uno solo. Puedes crear operadores propios con `infixl`, `infixr` o `infix`.

Evaluación perezosa

Es una estrategia de evaluación en la que una expresión no se reduce hasta que su valor es demandado por el contexto. Haskell implementa esta estrategia mediante thunks (expresiones pendientes almacenadas en el heap) y call-by-need (evaluación con memorización del

Escaneado con CamScanner